

2. OpenGL

Before starting our journey we should first define what OpenGL actually is. OpenGL is mainly considered an API (an **Application Programming Interface**) that provides us with a large set of functions that we can use to manipulate graphics and images. However, OpenGL by itself is not an API, but merely a specification, developed and maintained by the www.khronos.org/.



The OpenGL specification specifies exactly what the result/output of each function should be and how it should perform. It is then up to the developers *implementing* this specification to come up with a solution of how this function should operate. Since the OpenGL specification does not give us implementation details, the actual developed versions of OpenGL are allowed to have different implementations, as long as their results comply with the specification (and are thus the same to the user).

The people developing the actual OpenGL libraries are usually the graphics card manufacturers. Each graphics card that you buy supports specific versions of OpenGL which are the versions of OpenGL developed specifically for that card (series). When using an Apple system the OpenGL library is maintained by Apple themselves and under Linux there exists a combination of graphic suppliers' versions and hobbyists' adaptations of these libraries. This also means that whenever OpenGL is showing weird behavior that it shouldn't, this is most likely the fault of the graphics cards manufacturers (or whoever developed/maintained the library).

Since most implementations are built by graphics card manufacturers. Whenever there is a bug in the implementation this is usually solved by updating your video card drivers; those drivers include the newest versions of OpenGL that your card supports. This is one of the reasons why it's always advised to occasionally update your graphic drivers.

Khronos publicly hosts all specification documents for all the OpenGL versions. The interested reader can find the OpenGL specification¹ of version 3.3 (which is what we'll be using), which is a good read if you want to delve into the details of OpenGL (note how they mostly just describe results and not implementations). The specifications also provide a great reference for finding the **exact** workings of its functions.

2.1 Core-profile vs Immediate mode

In the old days, using OpenGL meant developing in **immediate mode** (often referred to as the **fixed function pipeline**) which was an easy-to-use method for drawing graphics. Most of the functionality of OpenGL was hidden inside the library and developers did not have much control over how OpenGL does its calculations. Developers eventually got hungry for more flexibility and over time the specifications became more flexible as a result; developers gained more control over their graphics. The immediate mode is really easy to use and understand, but it is also extremely inefficient. For that reason the specification started to deprecate immediate mode functionality from version 3.2 onwards and started motivating developers to develop in OpenGL's **core-profile** mode, which is a division of OpenGL's specification that removed all old deprecated functionality.

When using OpenGL's core-profile, OpenGL forces us to use modern practices. Whenever we try to use one of OpenGL's deprecated functions, OpenGL raises an error and stops drawing. The

¹ www.opengl.org/registry/doc/glspec33.core.20100311.withchanges.pdf

advantage of learning the modern approach is that it is very flexible and efficient. However, it's also more difficult to learn. The immediate mode abstracted quite a lot from the **actual** operations OpenGL performed and while it was easy to learn, it was hard to grasp how OpenGL actually operates. The modern approach requires the developer to truly understand OpenGL and graphics programming and while it is a bit difficult, it allows for much more flexibility, more efficiency and most importantly: a much better understanding of graphics programming.

This is also the reason why this book is geared at core-profile OpenGL version 3.3. Although it is more difficult, it is greatly worth the effort.

As of today, higher versions of OpenGL are available to choose from (at the time of writing 4.6) at which you may ask: why do I want to learn OpenGL 3.3 when OpenGL 4.6 is out? The answer to that question is relatively simple. All future versions of OpenGL starting from 3.3 add extra useful features to OpenGL without changing OpenGL's core mechanics; the newer versions just introduce slightly more efficient or more useful ways to accomplish the same tasks. The result is that all concepts and techniques remain the same over the modern OpenGL versions so it is perfectly valid to learn OpenGL 3.3. Whenever you're ready and/or more experienced you can easily use specific functionality from more recent OpenGL versions.

When using functionality from the most recent version of OpenGL, only the most modern graphics cards will be able to run your application. This is often why most developers generally target lower versions of OpenGL and optionally enable higher version functionality.

In some chapters you'll find more modern features which are noted down as such.

2.2 Extensions

A great feature of OpenGL is its support of extensions. Whenever a graphics company comes up with a new technique or a new large optimization for rendering this is often found in an **extension** implemented in the drivers. If the hardware an application runs on supports such an extension the developer can use the functionality provided by the extension for more advanced or efficient graphics. This way, a graphics developer can still use these new rendering techniques without having to wait for OpenGL to include the functionality in its future versions, simply by checking if the extension is supported by the graphics card. Often, when an extension is popular or very useful it eventually becomes part of future OpenGL versions.

The developer has to query whether any of these extensions are available before using them (or use an OpenGL extension library). This allows the developer to do things better or more efficient, based on whether an extension is available:

```
if(GL_ARB_extension_name)
{
    // Do cool new and modern stuff supported by hardware
}
else
{
    // Extension not supported: do it the old way
}
```

With OpenGL version 3.3 we rarely need an extension for most techniques, but wherever it is necessary proper instructions are provided.

2.3 State machine

OpenGL is by itself a large state machine: a collection of variables that define how OpenGL should currently operate. The state of OpenGL is commonly referred to as the OpenGL *context*. When using OpenGL, we often change its state by setting some options, manipulating some buffers and then render using the current context.

Whenever we tell OpenGL that we now want to draw lines instead of triangles for example, we change the state of OpenGL by changing some context variable that sets how OpenGL should draw. As soon as we change the context by telling OpenGL it should draw lines, the next drawing commands will now draw lines instead of triangles.

When working in OpenGL we will come across several *state-changing* functions that change the context and several *state-using* functions that perform some operations based on the current state of OpenGL. As long as you keep in mind that OpenGL is basically one large state machine, most of its functionality will make more sense.

2.4 Objects

The OpenGL libraries are written in C and allows for many derivations in other languages, but in its core it remains a C-library. Since many of C's language-constructs do not translate that well to other higher-level languages, OpenGL was developed with several abstractions in mind. One of those abstractions are *objects* in OpenGL.

An *object* in OpenGL is a collection of options that represents a subset of OpenGL's state. For example, we could have an object that represents the settings of the drawing window; we could then set its size, how many colors it supports and so on. One could visualize an object as a C-like struct:

```
struct object_name {
    float option1;
    int option2;
    char[] name;
};
```

Whenever we want to use objects it generally looks something like this (with OpenGL's context visualized as a large struct):

```
// The State of OpenGL
struct OpenGL_Context {
    ...
    object_name* object_Window_Target;
    ...
};
```

```
// create object
unsigned int objectId = 0;
glGenObject(1, &objectId);
// bind/assign object to context
glBindObject(GL_WINDOW_TARGET, objectId);
// set options of object currently bound to GL_WINDOW_TARGET
glSetObjectOption(GL_WINDOW_TARGET, GL_OPTION_WINDOW_WIDTH, 800);
glSetObjectOption(GL_WINDOW_TARGET, GL_OPTION_WINDOW_HEIGHT, 600);
// set context target back to default
glBindObject(GL_WINDOW_TARGET, 0);
```

This little piece of code is a workflow you'll frequently see when working with OpenGL. We first create an object and store a reference to it as an id (the real object's data is stored behind the